

SIS TEAM	Version : V1.1	 UNIVERSITÉ DE LORRAINE	
PR40_FO001	Date : 04/10/2022	 	
	Type : Formulaire	Date d'application :	
	Référence : PR5X_FO002	Version : V1.1	
		Nombre de page (s) : 3 pages	
PR 52 Formulaire de dossier technique			
	Nom	Responsabilité	Date
Rédaction	MAURY Bruno DOGUET Martin	Consultant Senior Responsable scientifique	05/03/2025 05/03/2025
Validation	HEMAIDIA Riham VARIS Pierre	Secrétaire Générale Pilote PM1 « Adapter la stratégie »	
Accord du responsable qualité pour diffusion (COZIC Valentin)			

I . Objet et domaine d'application

Objectif : Ce formulaire, destiné aux consultants de SIS TEAM, décrit les opérations à réaliser pour élaborer un dossier technique final de leur projet, permettant une reproduction complète du livrable avec la garantie de performances conformes au cahier des charges initial.

Ce dossier technique doit comporter toutes les étapes et informations nécessaires, avec les références des composants, des logiciels et bibliothèques avec leur version, logigrammes, schémas techniques et plans de conception.

Tous les fichiers de code (commentés) et de fabrication seront joints en annexe.

Mots clés : Cahier des charges, Suivi de projet, Livrable, Dossier technique

II . Responsabilités

La mise à jour de cette procédure est sous la responsabilité du responsable du macroprocessus PR4 : Réaliser et livrer le projet.

Sa mise en œuvre est sous la responsabilité du comité de pilotage de SIS TEAM à l'issue de la livraison du cahier des charges, du dossier découpage projet.

L'élaboration et la mise à jour des POS et formulaires associés est sous la responsabilité de l'ensemble des collaborateurs de SIS TEAM.

La validation des documents qualité est sous la responsabilité des pilotes du processus PM3 « Piloter l'amélioration continue ». Une validation par mail peut être acceptée.

III . Documents de référence

Référentiel d'organisation : *ISO9001 : 2015 Systèmes de management de la qualité - Exigences*
Référentiel planification, support (faisabilité et ressources) : *NF EN 16271* *Cahier des charges*

IV . Historique des modifications

Date	Version	Description	Rédacteur(s)
05/03/2025	00	Projet	B. Maury

V . Formulaire

Titre du projet	Estimation de la PaO2 depuis la Spo2.
Client	Georgin Didier
Prestataires STN	Bah Fatoumata Binta et Zakrzewski Nathan Encadrante : Elisa Quintino

V . 1 . Cadrage du projet

- Cahier des charges

Définition globale du besoin

Didier Georgin, ingénieur biomédical au centre hospitalier d'Epinal, demande une mesure ou une estimation de la VO2Max (consommation maximale d'oxygène) sur piste extérieure, pour un sujet sportif, ayant préalablement fait un test d'effort sous le contrôle d'un médecin. Il se demande également s'il n'existe pas un lien exploitable entre la fréquence respiratoire et la VO2Max

Pour notre projet on nous a alors demandé :

- Mise en œuvre d'un capteur SpO2 adaptée à une utilisation durant une activité sportive.
- Faire le lien entre la SpO2 et la PaO2.
- Récupération de la fréquence respiratoire.

Périmètre

- Notre projet se limite à l'estimation de la PaO2 grâce à la SpO2, que nous aurons depuis un dispositif médical, et à la récupération de la fréquence cardiaque.
- Le projet devra être adapté à la pratique sportive haute intensité avec parfois un environnement compliqué.
- Il ne doit pas empiéter sur les autres projets SIS TEAM Nancy qui peuvent être liés à la fréquence cardiaque par exemple. Nous devons cependant collaborer avec les autres projets pour capitaliser les informations.

Définitions des fonctions à assurer et contraintes à respecter, avec les EMT

En fin de projet notre dispositif répond au cahier des charges suivant :

- Mesure de la SpO2 Fiable à l'aide d'un capteur d'oxymétrie de pouls au doigt.
- Mesure de la fréquence pulsatile à l'aide d'un capteur d'oxymétrie de pouls au doigt
- Mesures de paramètres environnementaux : Humidité et température.
- Estimer la PaO2 à l'aide d'un modèle mathématique fiable.
- Affichage des données en local sur un écran OLED.
- Envoie de l'ensemble des données en LoRa sur une centrale.
- Créer un système portatif et ergonomique adaptable à une utilisation sportive.
- Respecter les normes en vigueur sur les systèmes d'oxymétrie de pouls.
- Mettre en place un protocole d'utilisation.

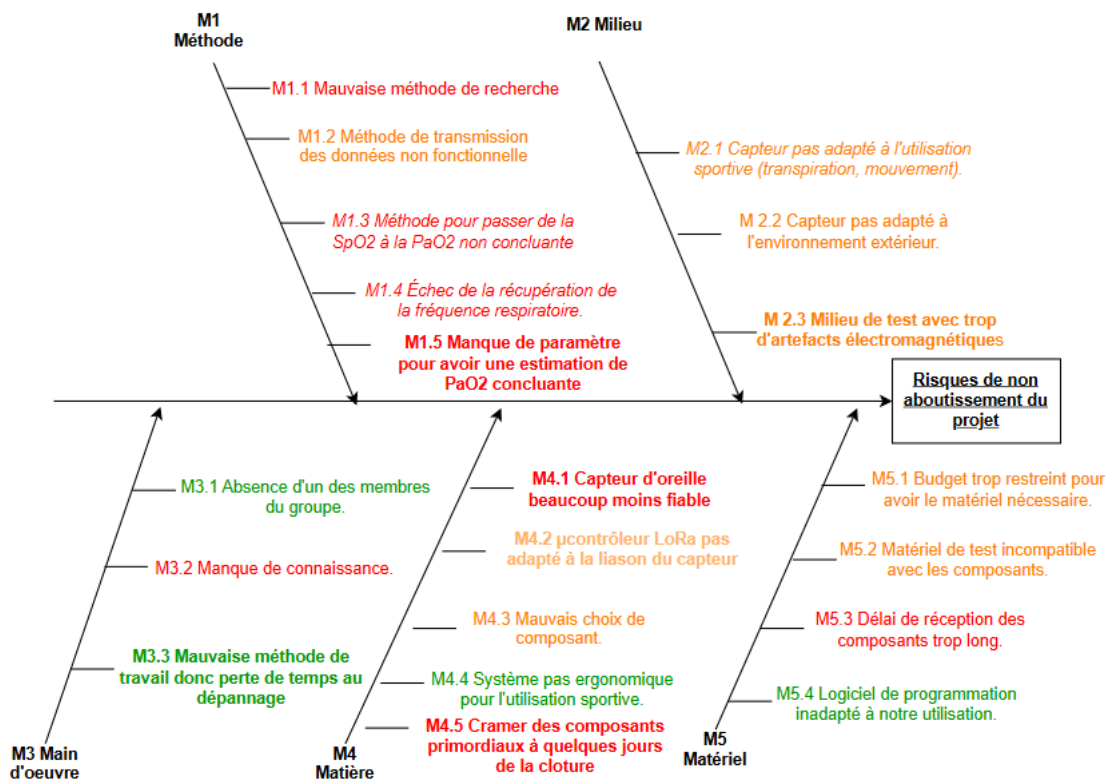
Les normes nous donnent un écart toléré de 2% sur la valeur de la Spo2 et de 3 battements sur la valeur de la fréquence pulsatile.

Contexte réglementaire et normatif

- Le règlement 2017/745 relatif aux dispositifs médicaux.

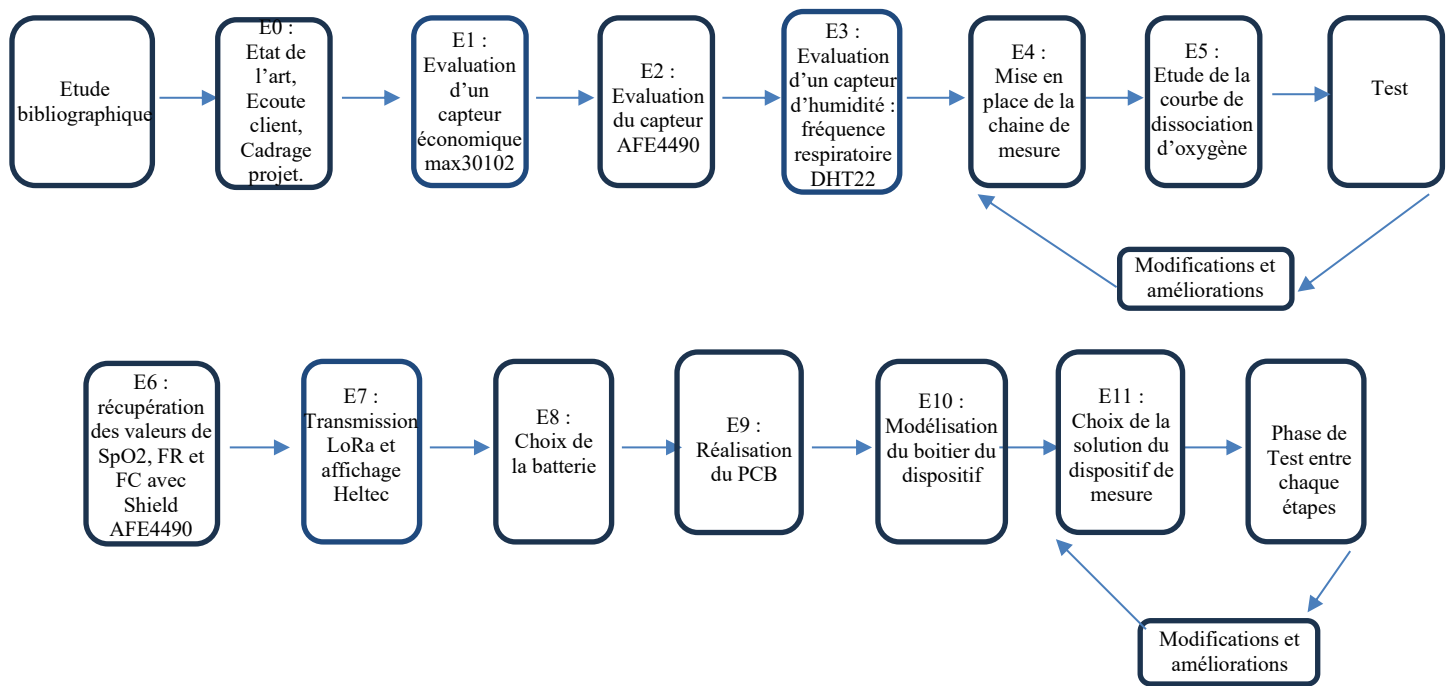
- Normes ISO 13485 liée aux dispositifs médicaux et ISO 80601-2-61 liée à également la qualité ISO 9001.
- Sécurité électrique : CEI 60601-1

Analyse de risques projet (version finale)

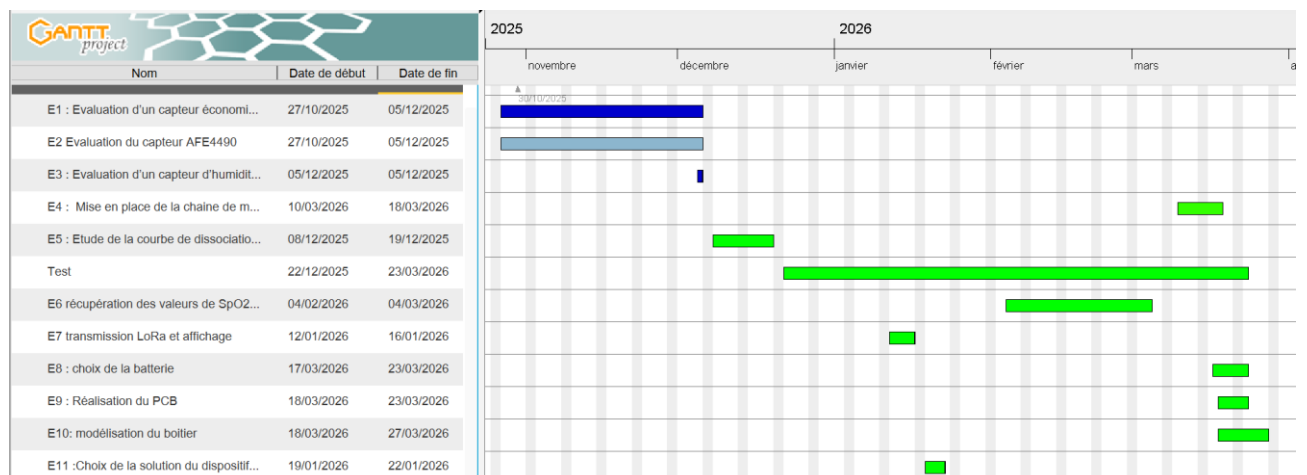


Les éléments en *italique* sont les éléments présents initialement qui se sont réalisés.
Les éléments en **gras** sont les éléments rajoutés en fin de projet.

- Découpage projet final

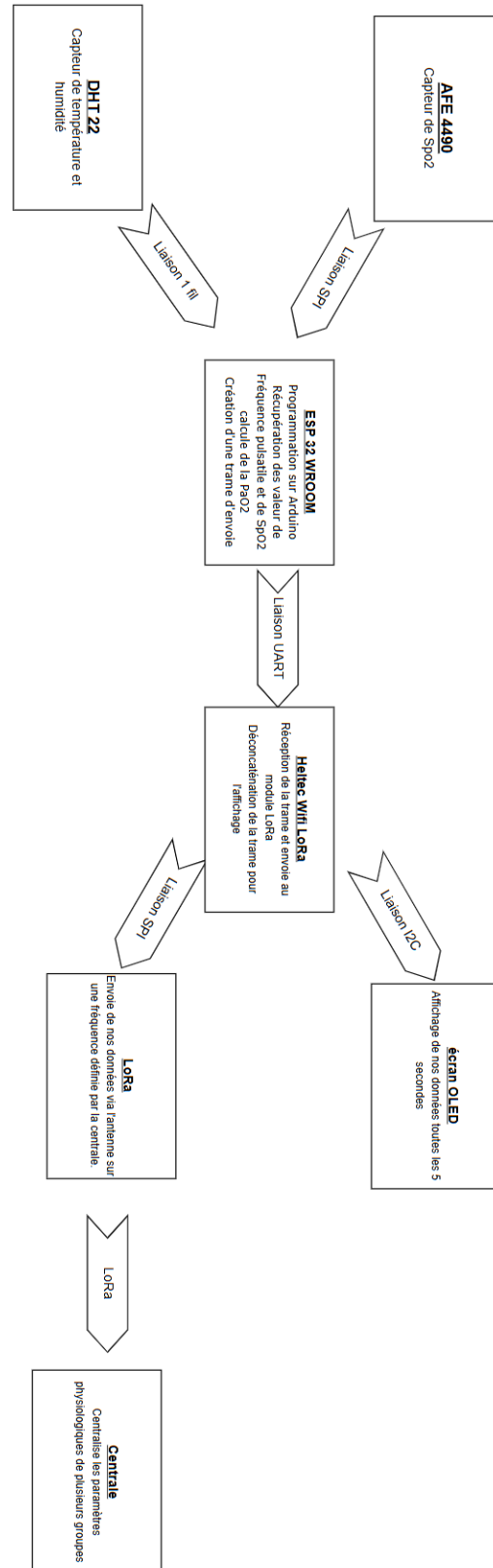


- Diagramme de Gantt effectif



V . 2 . Solutions techniques

- Schéma de principe ou relationnel commenté (*le schéma peut être redécoupé par sous parties pour en faciliter la compréhension*). Un découpage fonctionnel ou Bloc Diagramme Fonctionnel peut être utile en amont pour organiser la description.



- Liste des composants et éléments nécessaires avec leurs spécifications

- Shield AFE4490 Protocentrale, Capteur oxymétrie de pouls

- Liaison SPI
- Alimentation 5V
- Brochage DB9 pour pince Neilcor



- Pince Neilcore SpO2

- Oreille et Doigt



- Heltec Wifi LoRa V3

- Ecran OLED et communication LoRa
- USB Type C
- Alimentation 3V3 ou 5V ou 3,7V(batterie)



- ESP 32 Wroom DevKit V1

- Alimentation entre 4,5 et 12V
- Broche SPI
- Micro USB



- DHT22 Capteur de température et d'humidité

- Alimentation entre 3,3 et 6V
- Signal digital via un fil



- LM7805

- Régulateur de tension sortie 5V
- Entrée entre 7 et 25V



- Batterie Quzmo

- 9V
- Rechargeable
- 1300mA

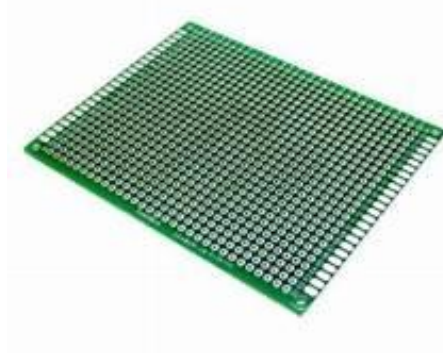


- Interrupteur R13-73

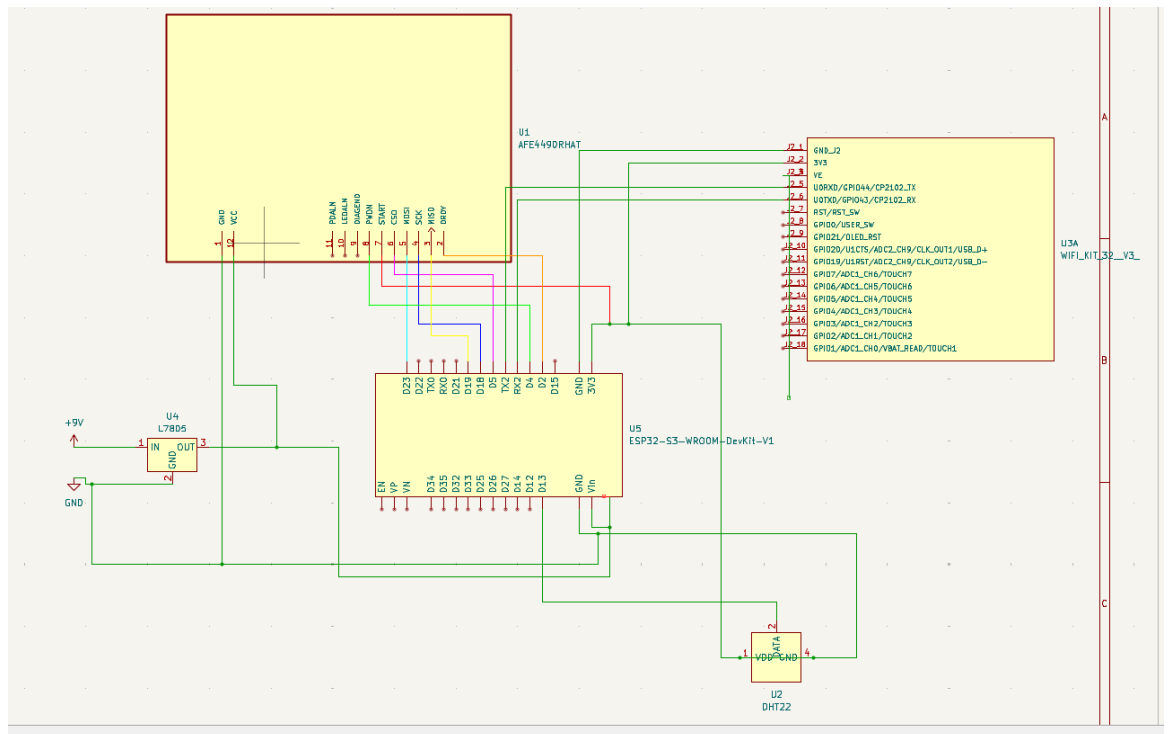


- Schémas de conception du PCB (Kicad ou autre) et d'implantation des composants.

Pour réaliser notre PCB, nous n'avons pas utilisé le logiciel Kicad. Nous avons directement soudé sur une plaque faite pour cet effet. Comme celle-ci-dessous en 7*9 cm.



Nous avons alors réalisé un schéma électronique et un tableau d'implantation des composants :



Broche	Implantation	Broche	Implantation	Broche	implantation
AFE_GND	QT	WROOM_gnd	AB	LM7805_gnd	VE
AFE_DRDY	QU	WROOM_d2	AD		
AFE_MISO	QV	WROOM_d19	AJ		
AFE_SCK	QW	WROOM_d18	AI		
AFE_MOSI	QX	WROOM_d23	AO		
AFE_CS0	QY	WROOM_d5	AH		
AFE_START	QZ	WROOM_3V3	AA	DHT_+	ZA
AFE_PWDN	QA	WROOM_d4	AE		
AFE_VCC	QE	WROOM_Vin	KA	LM7805_out	UE
Heltec_3V3	ZC	WROOM	AA		
Heltec_GND	QA	WROOM	KB		
Heltec_4	ZM	WROOM	AG		
Heltec_45	ZF	WROOM	AF		
DHT_-	ZW	WROOM			
DHT_Data	ZY	WROOM			
Batterie +		LM7805_in	WE		
Batterie -					

Les broches placées sur la même ligne sont les broches reliées par des fils.

- L'ensemble des schémas seront fournis en annexe en pdf avec une cartouche référençant le projet et ses auteurs, la date, ainsi que « © SIS TEAM NANCY »

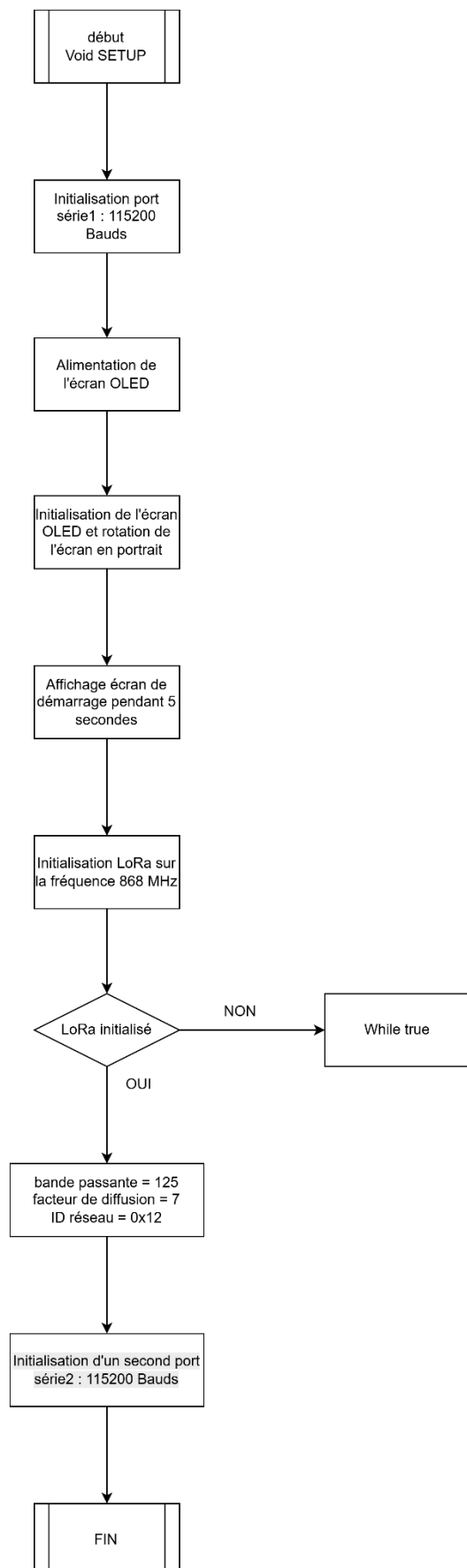
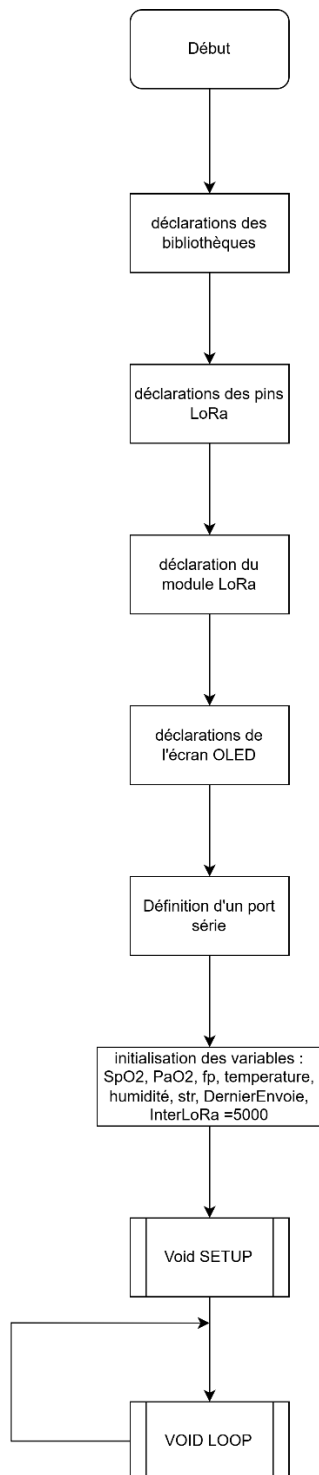
Annexes 1 schéma électronique

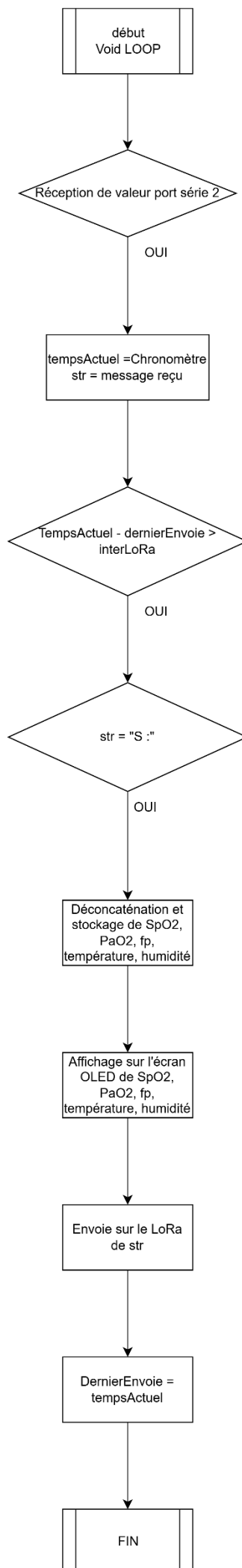
Pas de dossier de conception.

V . 3 . Solutions logicielles

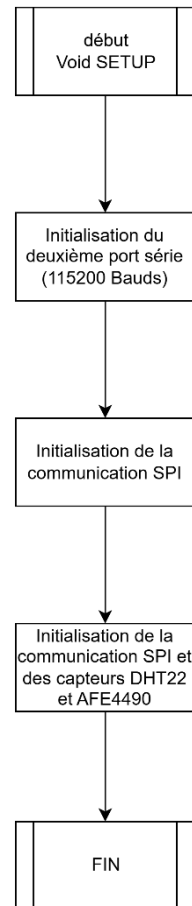
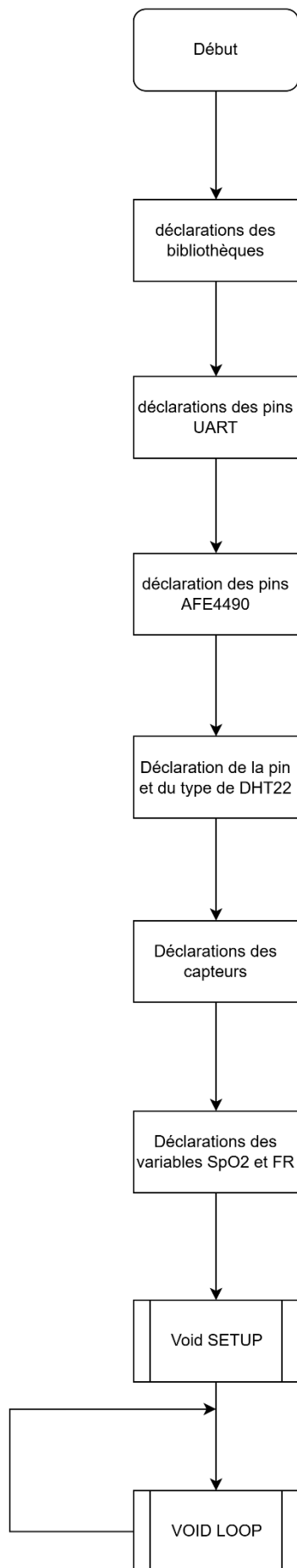
- Logigrammes commentés

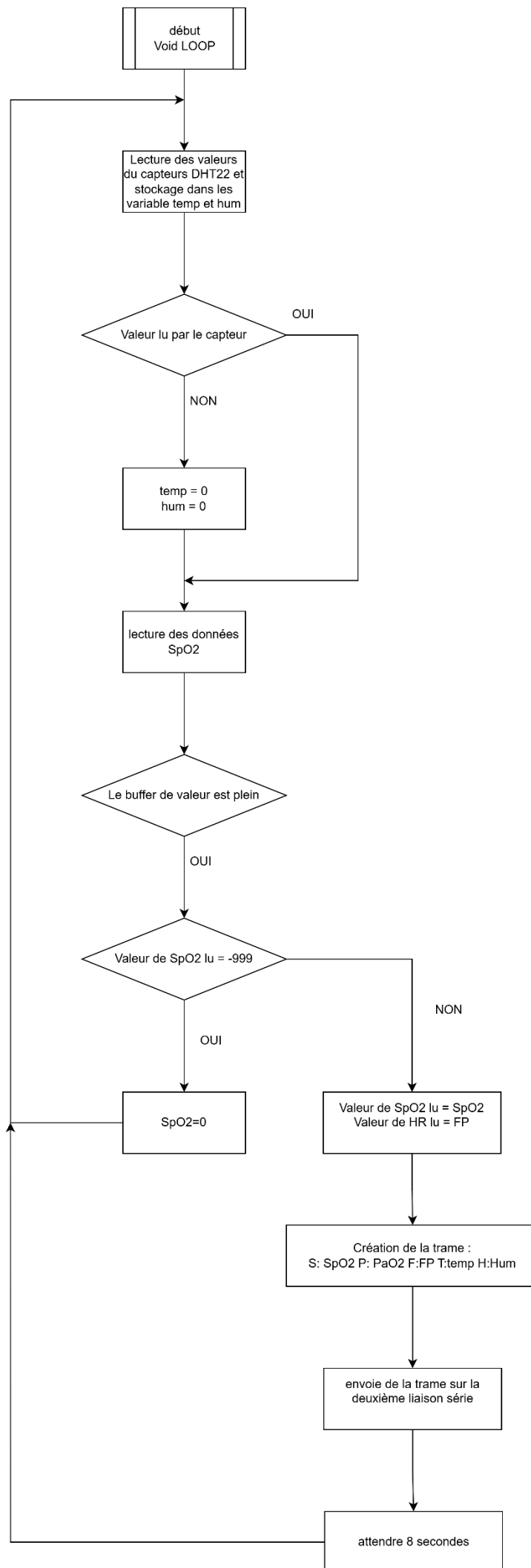
Logigramme Heltec





Logigramme Wroom





Le code du Heltec nous permet d'envoyer les différents paramètres mesurés par notre système. Tout d'abord il va déclarer chose :

- Les bibliothèques
- Les pins LoRa, qui sont câblés en interne.
- Le module LoRa, qui va permettre la transmission à la centrale.
- L'écran OLED, pour l'affichage des valeurs en local.
- Le port série, pour la réception du Wroom.
- Les variable, SpO2, FR, etc....

Dans le SETUP, on va ensuite mettre en route tous ces modules, régler leurs paramètres et afficher un écran de présentation.

Pour finir, dans le LOOP, On va d'abord vérifier qu'on reçoit des bonnes valeurs sur le deuxième port série. Si oui, on va envoyer exactement le même message sur l'antenne LoRa pour que la centrale puisse recevoir. Nous allons en parallèle déconcaténer chaque valeur pour les afficher en local. La réception va être temporisée toutes les 5 secondes grâce à un chronomètre.

Le code du Wroom quant à lui sert à l'acquisition des données des capteurs (DHT22 et AFE4490). Il sert de pont entre les capteurs et le Heltec.

Comme pour le Heltec, on commence par une étape de déclaration :

- Des bibliothèques
- Des pins pour la liaison série, envoi des données au Heltec.
- Des capteurs et de leurs pins, AFE 4490 et DHT22
- Des variables

Dans le SETUP, nous allons démarrer les deux liaisons, série pour le Heltec et SPI pour le capteur AFE4490.

Dans le LOOP du Wroom, nous allons commencer par lire et vérifier les valeurs de température et d'humidité. Si les valeurs ne montrent pas d'erreur, elles sont stockées dans les variables correspondantes. Nous allons ensuite lire les valeurs de SpO2. Dans la bibliothèque ProtoCentrale, il existe une gestion d'erreur pour ne pas stocker des valeurs quand il n'y a pas de doigt. Quand il y a un doigt les valeurs sont stockées dans les variables correspondantes. Pour finir, une trame est créée, on va concaténer toutes nos valeurs en une chaîne de caractère et l'envoyer par la communication série toutes les 8 secondes.

- Liste et versions des environnements et langages de programmation, drivers, compléments, bibliothèques, logiciels complémentaires nécessaires

Logiciel de programmation :

- Arduino

Langages de programmation :

- C++ simplifié

Bibliothèques :

- **RadioLib** by Jon Gromes (communication LoRa) (Heltec)
- **ESP8266 and ESP32 OLED driver for SSD1306 displays** by ThingPulse, Fabrice Weinberg

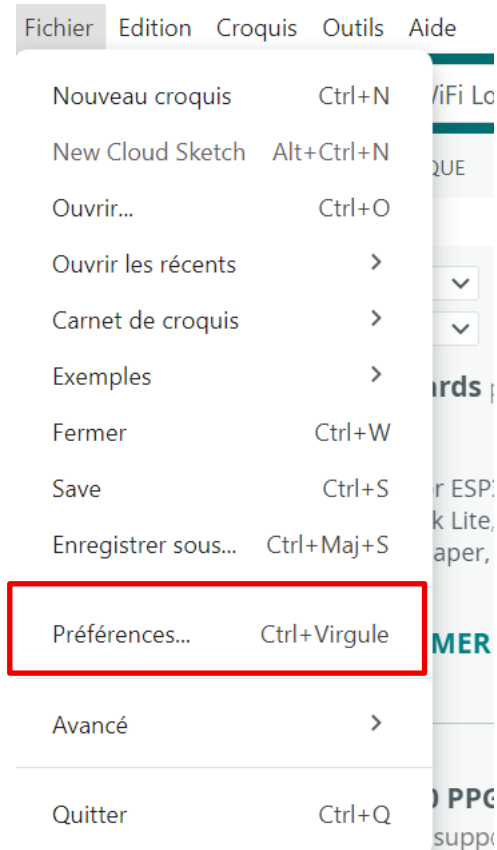
- **DHT sensor library** by Adafruit
- **Heltec ESP32 Dev-Board** by Heltec Automation
- **ProtoCentral AFE4490 PPG and SpO2 board library** ProtoCentrale (télécharger le ZIP sur le Github ProtoCentrale)
- Sources externes référencées (si possible GitHub, source d'origine)

<https://github.com/Protocentral/protocentral-afe4490-arduino>

- Modes opératoires d'installation avec le matériel ou logiciels annexes nécessaires et leurs versions/spécifications. Par exemple pour l'installation d'une base en client-serveur : *spécifications du serveur, spécification du poste client, installation des applications, déclarations des liens, configuration des droits sur le poste client . .)*

Obtenir les cartes ESP 32 sur Arduino

- 1) Dans arduino IDE, allez dans Fichier>Préférences



2) Dans le champ URL Coller :

```
https://raw.githubusercontent.com/expressif/arduino-esp32/gh-  
pages/package_esp32_index.json,  
http://arduino.esp8266.com/stable/package_esp8266com_index.json
```

The screenshot shows the 'Préférences' (Preferences) dialog box in the Arduino IDE, with the 'Paramètres' (Parameters) tab selected. The 'URL de gestionnaire de cartes supplémentaires' (Additional Board Manager URLs) field is highlighted with a red rectangle. The field contains the following text:

```
https://raw.githubusercontent.com/expressif/arduino-esp32/gh-  
pages/package_esp32_index.json,  
http://arduino.esp8266.com/stable/package_esp8266com_index.json
```

Other visible settings include:

- Localisation du croquis: c:\Users\sylvi\OneDrive\Documents\Arduino
- Taille de police de l'éditeur: 14
- Échelle de l'interface: Automatique 100 %
- Thème de couleur: Clair
- Langage de l'éditeur: français (Reload required)
- Afficher la sortie de débogage verbeuse pendant: compiler, téléverser
- Avertissements du compilateur: Aucun
- Enregistrement automatique: [checked]
- Suggestion rapide pour l'éditeur: [unchecked]

Buttons at the bottom: ANNULER, OK.

Puis cliquer sur OK

3) Ouvrez le gestionnaire de cartes et télécharger : **esp32** by Expressif Systems

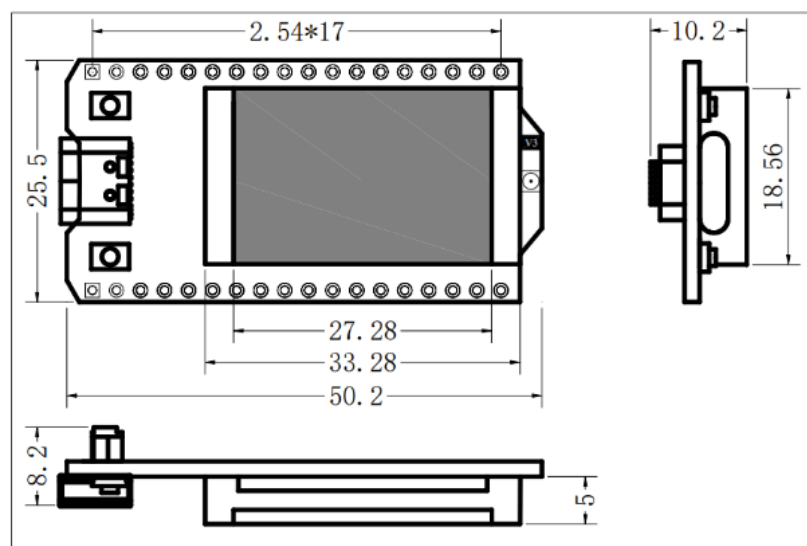


Les fichiers de code et les applications réalisées seront envoyés à Sis team Nancy en même temps que ce dossier technique.

V . 4 . Solutions mécaniques

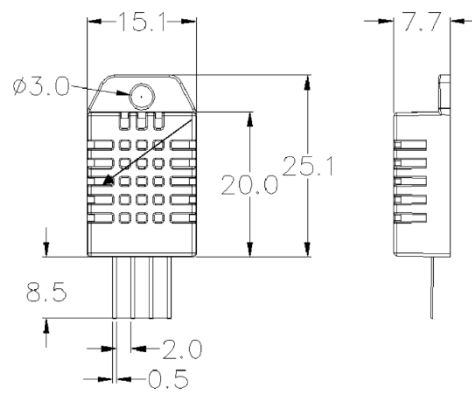
- Contraintes mécaniques initiales (dimensions, matériaux) pour les pièces et boîtiers

Dimensions du Heltec :

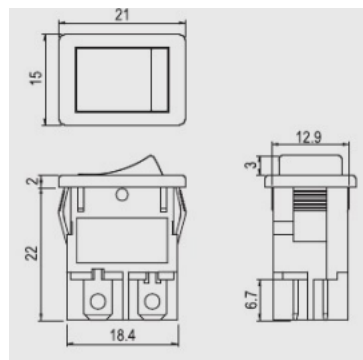


Dimension pour le passage de l'antenne : 6,6 mm

Dimensions du DHT 22 :

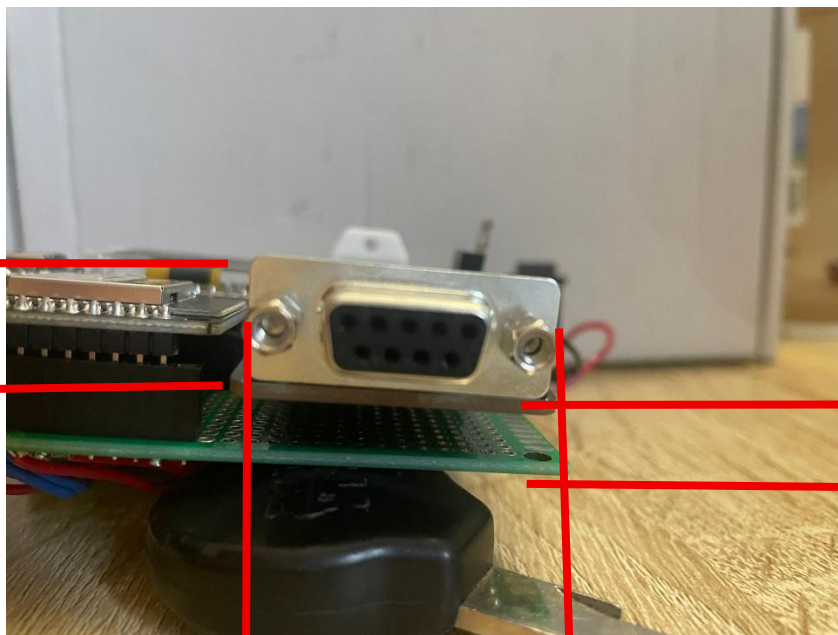


Dimensions de l'interrupteur :



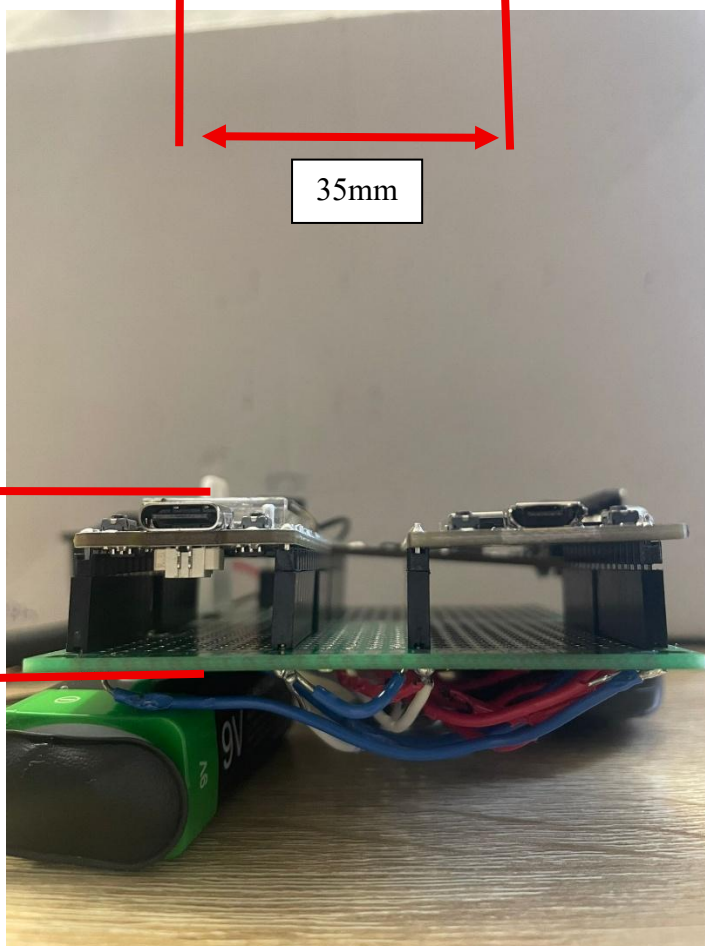
Dimensions PCB : 7*9 pour la plaque

13,9mm



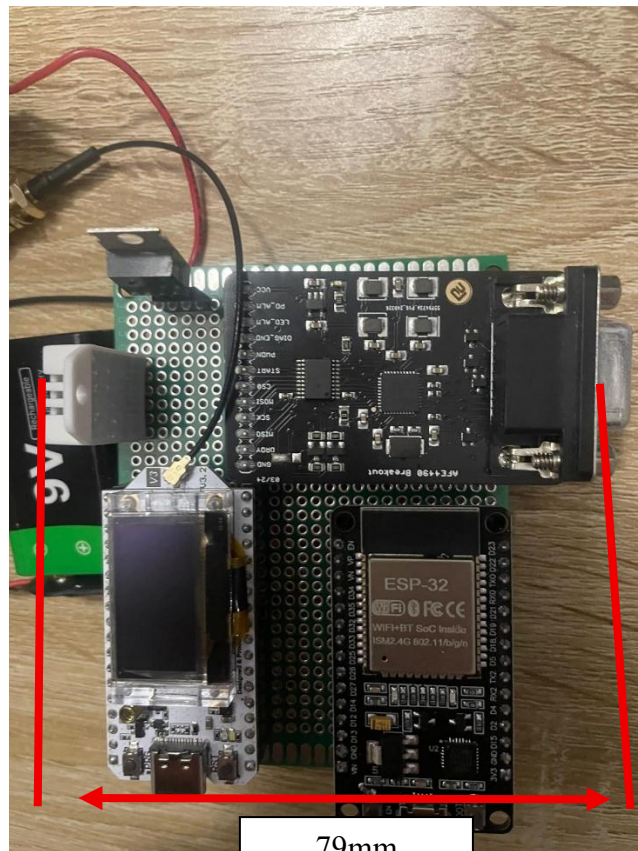
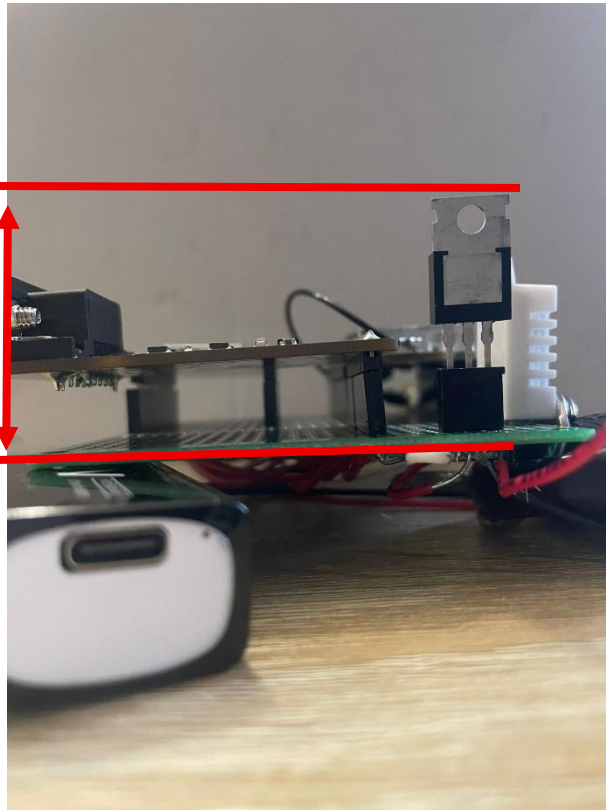
14mm

35mm



20mm

31,9mm



- Plans descriptifs avec les cotations (plusieurs vues)
Nous n'avons pas eu de boîtier final donc pas de plan descriptif final.
- L'ensemble des plans seront fournis en annexe en pdf avec un cartouche référençant le projet et ses auteurs, la date, ainsi que « © SIS TEAM NANCY »

Les fichiers de conception seront envoyés à Sis team Nancy en même temps que ce dossier technique.

V . 5 . Contrôle de performance

Nos EMT n'ont pas été réalisés car les valeurs de fréquence cardiaque sont très variables et ne sont pas fiables. Notre SpO2 est aussi très sensible aux mouvements et peut être très différente selon le type de peau.

V . 6 . Mode d'emploi

Nous n'avons pas eu besoin de préparer de mode d'emploi, un simple allumage de l'interrupteur permet de faire fonctionner notre dispositif.

V . 7 . Déroulement du projet et ouvertures

Le projet dans son ensemble s'est bien passé et nous a permis d'apprendre beaucoup de choses, que ce soit sur la partie technique, recherche et gestion de projet. On a pu apprendre comment se déroule les différentes étapes de la mise sur pied d'un dispositif médical.

Au vu des résultats atteints en cette fin de projet, nos observations ainsi que les remarques des consultants nous permettent d'affirmer que de nombreux points peuvent encore être améliorés. Le livrable n'est clairement pas abouti et il reste encore du travail.

Tout d'abord, concernant le livrable final, un véritable prototype de PCB pourrait être conçu sur KiCad afin de faciliter la capitalisation et la reproduction de notre dispositif. Cela permettrait une production à plus grande échelle que la solution de soudure manuelle que nous avons utilisée. Il serait également possible de proposer une modélisation plus esthétique et ergonomique que la solution actuelle, en s'appuyant sur des plans clairs et sûrs.

Pour la suite, de belles perspectives peuvent être envisagées pour la solution elle-même. Pour commencer, notre estimation de la PaO2 actuelle présente une incertitude due au modèle mathématique choisi. En effet, dans notre équation, la P50 (valeur de PaO2 à 50 % de SpO2) varie selon le pH sanguin (entre 7,2 et 7,6). Il faudrait donc, à l'avenir, une solution prenant en considération ce paramètre.

Concernant l'envoi LoRa, grâce à la bibliothèque ProtoCentral de l'AFE, il est possible d'extraire les données brutes de la courbe de SpO2. On pourrait alors communiquer ces valeurs à une centrale pour afficher les courbes et créer un véritable moniteur multiparamétrique.

Sur la prise de la SpO2, un capteur d'oreille serait probablement plus adapté pour éviter les mouvements en condition sportive. Malheureusement, avec la solution actuelle, les valeurs ne sont pas assez fiables ; cela constitue donc un axe d'amélioration. Enfin, une étude métrologique du système est nécessaire pour vérifier que notre méthode est stable et fiable dans le temps.

Nous aurions voulu approfondir le travail sur le capteur de SpO2, mais la première phase de recherche sur les différentes technologies nous a malheureusement demandé énormément de temps.

VI . Validation du dossier technique :

Remarques :

Date :

Nom du validant :
(Responsable suivi de projet et/ou tuteur)

Signature :

VII . ANNEXES

-----Code Wroom ESP32-----

```
////This code was created by two Master's Year 1 biomedical engineering students, ////  
////as part of a project supervised by the junior enterprise, Sisteam Nancy.////  
////STN_11: "Estimation of PaO2 from SpO2" BAH Fatoumata Binta, ZAKRZEWSKI Nathan////  
//////////////////////////////////// ©SISTEAM Nancy 2026////////////////////////////////////
```

```
// Library declarations
```

```
#include <SPI.h>
```

```
#include "protocentral_afe44xx.h"
```

```
#include <DHT.h>
```

```
// UART pin definitions
```

```
#define HELTEC_RX 16
```

```
#define HELTEC_TX 17
```

```
// AFE4490 pin definitions
```

```
#define AFE44XX_CS_PIN 5
```

```
#define AFE44XX_PWDN_PIN 4
```

```
#define AFE44XX_INTNUM 0
```

```
// DHT22 pin definitions
```

```
#define DHTPIN 13
```

```
#define DHTTYPE DHT22
```

```
// Sensor and variable declarations
```

```
AFE44XX afe44xx(AFE44XX_CS_PIN, AFE44XX_PWDN_PIN);
```

```
afe44xx_data afe44xx_raw_data;
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
int32_t heart_rate_prev=0;
```

```
int32_t spo2=0;
```

```
// PAO2 calculation
```

```
float pao2=0;
```

```
float n=0.7;
```

```
float p50=26.6;
```

```
void setup() {
```

```
// Serial communication setup
```

```
Serial2.begin(115200, SERIAL_8N1, HELTEC_RX, HELTEC_TX);
```

```
// Initialization of sensors and SPI communication
```

```
Serial.begin(115200);
```

```
Serial.println("Intilaziting AFE44xx.. ");
```

```
SPI.begin();
```

```
afe44xx.afe44xx_init();
```

```
Serial.println("Inited...");
```

```

    dht.begin();
}

void loop() {

// Reading data from the DHT22 sensor
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

// DHT11 error handling
    if (isnan(temperature) || isnan(humidity)) {
        temperature = 0;
        humidity = 0;
    }
// Reading raw data from the AFE4490 sensor
    afe44xx.get_AFE44XX_Data(&afe44xx_raw_data);

// AFE4490 error handling
    if (afe44xx_raw_data.buffer_count_overflow)
    {
        if(afe44xx_raw_data.spo2 == -999)
        {
            spo2=0;
            Serial.println("-1");
        }
    }
// Storing data into variables
    else if ((heart_rate_prev != afe44xx_raw_data.heart_rate) || (spo2 != afe44xx_raw_data.spo2))
    {
        heart_rate_prev = afe44xx_raw_data.heart_rate;
        spo2 = afe44xx_raw_data.spo2;
        pao2= 0 ;//p50*(spo2/(1-spo2))^(1/n);

// Frame creation
        String message = "S:" + String(spo2) +
            " P:" + String(pao2) +
            " F:" + String(heart_rate_prev) +
            " T:" + String(temperature) +
            " H:" + String(humidity);

// Send to Heltec every 8 seconds
        Serial2.println(message);
        delay(8);
    }
}
}

```


-----Code Heltec Wfi LoRa V3-----

```
////This code was created by two Master's Year 1 biomedical engineering students, ////
////as part of a project supervised by the junior enterprise, Sisteam Nancy.////////
////STN_11: "Estimation of PaO2 from SpO2" BAH Fatoumata Binta, ZAKRZEWSKI Nathan////
////////// ©SISTEAM Nancy 2026//////////
```

```
//Library declarations
#include <Arduino.h>
#include <RadioLib.h>
#include <Wire.h>
#include "HT_SSD1306Wire.h"
```

```
// LoRa pin definitions
#define LORA_SS 8
#define LORA_DIO1 14
#define LORA_RST 12
#define LORA_BUSY 13
```

```
// LoRa module declaration
Module loraModule(LORA_SS, LORA_DIO1, LORA_RST, LORA_BUSY);
SX1262 radio(&loraModule);
```

```
// OLED display declaration
SSD1306Wire oled(0x3c, 500000, SDA_OLED, SCL_OLED, GEOMETRY_128_64,
RST_OLED);
```

```
// Serial port definition
HardwareSerial MySerial(1);
#define RX_PIN 4
#define TX_PIN 5
```

```
// Medical variables declaration
int spo2 = 0;
int pao2 = 0;
int fp = 0;
int temperature = 0;
float humidity = 0;
String str = "";
```

```
// Timer variable
unsigned long dernierEnvoiLoRa = 0;
const long intervalleLoRa = 5000;
```

```
void setup() {
// Serial communication setup
Serial.begin(115200);
```

```
// Powering on the OLED display
```

```

pinMode(Vext, OUTPUT);
digitalWrite(Vext, LOW);
delay(100);

// OLED initialization and startup screen
oled.init();
oled.screenRotate(ANGLE_90_DEGREE);
oled.clear();
oled.setFont(ArialMT_Plain_10);
oled.drawString(0, 0, "Projet de :");
oled.drawString(0, 20, "Bah");
oled.drawString(0, 40, "Binta");
oled.drawString(0, 60, "Zakrzewski");
oled.drawString(0, 80, "Nathan");
oled.drawString(0, 100, "Master IB1");
oled.display();
delay(5000);

// LoRa initialization
oled.clear();
oled.drawString(0, 0, "Initialisation TX...");
oled.display();
int state = radio.begin(868.0); // LoRa startup with 868 MHz transmission frequency
if (state == RADIOLIB_ERR_NONE) {
    radio.setBandwidth(125.0); // Bandwidth
    radio.setSpreadingFactor(7); // Spreading factor
    radio.setSyncWord(0x12); // Network identification
} else {
    while(true);
}
// Serial communication setup
MySerial.begin(115200, SERIAL_8N1, RX_PIN, TX_PIN); // (baud_rate, frame_format(bits,
parity, stop), RX_pin, TX_pin)
}

void loop() {

// Receiving data on the serial port
if (MySerial.available()) {
    str = MySerial.readStringUntil('\n');
    str.trim();

// Restarting the timer for LoRa transmission every 5 seconds
unsigned long tempsActuel = millis();
if (tempsActuel - dernierEnvoiLoRa >= intervalleLoRa) {
// Frame parsing for OLED display
if (str.startsWith("S:")) {
    spo2 = str.substring(str.indexOf("S:")+2, str.indexOf(" P:")).toFloat();
    pao2 = str.substring(str.indexOf("P:")+2, str.indexOf(" F:")).toFloat();
    fp = str.substring(str.indexOf("F:")+2, str.indexOf(" T:")).toFloat();
    temperature = str.substring(str.indexOf("T:")+2, str.indexOf(" H:")).toFloat();

```

```

humidity = str.substring(str.indexOf("H:")+2, str.indexOf(" S:")).toFloat();

if (spo2 <= 0) {
  oled.clear();
  oled.drawString(0, 0, "doigt absent");
  oled.drawString(0, 12, "PAO2 SPO2 FP");
  oled.drawString(0, 24, "absent");
  oled.drawString(15, 36, String(pao2) + " mmHg");
} else {
  oled.clear();
  oled.drawString(0, 0, "SPO2: " + String(spo2) + " %");
  oled.drawString(0, 12, "PAO2:");
  oled.drawString(15, 24, String(pao2) + " mmHg");
  oled.drawString(0, 36, "FP: " + String(fp) + " bpm");
}

// Displaying values on the OLED screen

oled.drawString(0, 48, "Temp:");
oled.drawString(15, 60, String(temperature) + " C");

oled.drawString(0, 72, "Hum:");
oled.drawString(15, 84, String(humidity) + " %");

oled.drawString(0, 104, "-----");

oled.drawString(0, 111, "envoi LoRa");
oled.display();}

if (str.length() > 0 && str.startsWith("S:")) {
  oled.drawString(0, 110, "ENVOI LORA...");
  oled.display();

// Sending data via LoRa
  int state = radio.transmit(str);

// Write to the serial port whether the transmission succeeded or not
  if (state == RADIOLIB_ERR_NONE) {
    Serial.println(F("[LoRa] Envoi réussi !"));
  } else {
    Serial.print(F("[LoRa] Erreur d'envoi : "));
    Serial.println(state);
  }
}

dernierEnvoiLoRa = tempsActuel; // Restart timer
}}
}

```

